

Project_MJS

시네마틱 시스템 구조 설명서

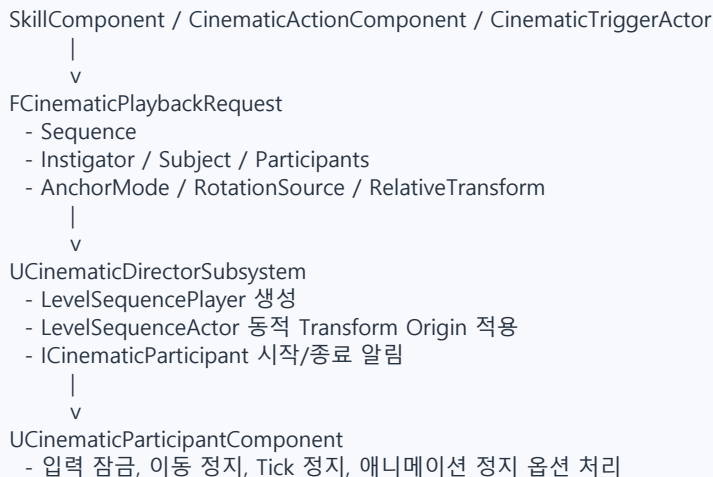
스킬 시퀀스, 궁극기 컷신, 일반 레벨 컷신을 하나의 Director 기반 재생 구조로 묶고, 플레이어 월드 위치/상대 위치/플레이어 회전/카메라 회전을 기준으로 동적 시퀀스를 재생하기 위한 구현 분석 문서입니다.

인코딩: UTF-8 / 출력: PDF / 폰트: Malgun Gothic / 기준 코드: Project_MJS Cinematic 폴더

핵심 결론

현재 구조는 전투 로직과 시네마틱 연출을 분리하는 방향이 맞습니다. 이번 업데이트로 Level Sequence를 에셋에 저장된 월드 위치에 고정하지 않고, 런타임에 플레이어, 타겟, 트리거, 카메라 방향을 기준으로 재배치할 수 있게 되었습니다. UE의 UDefaultLevelSequenceInstanceData.TransformOrigin을 사용하므로 시퀀서의 절대 Transform Section도 동적 원점 기준으로 평가됩니다.

1. 전체 구조



시네마틱 재생 요청자는 구체적인 입력 잠금 방식이나 AI 정지 방식을 몰라도 됩니다. 요청자는 FCinematicPlaybackRequest를 만들고 Director에 넘깁니다. Director는 시퀀스를 재생하고, 실제로 어떤 액터가 멈출지는 ICinematicParticipant 구현체가 독립적으로 처리합니다.

2. 이번 업데이트 요약

- FCinematicPlaybackRequest에 AnchorMode, RotationSource, AnchorActor, AnchorSocketName, TargetSocketName, RelativeTransform, ExplicitWorldTransform, ExplicitRotation, bUseYawOnly를 추가했습니다.
- FCinematicPlaybackContext에 AnchorWorldTransform과 bAppliedDynamicTransform을 추가해 참가자/디버그/VFX가 동일한 기준점을 읽을 수 있게 했습니다.
- UCinematicDirectorSubsystem이 LevelSequenceActor 생성 직후 동적 원점을 계산하고, UDefaultLevelSequenceInstanceData.TransformOrigin에 적용하도록 만들었습니다.

- UCinematicActionComponent에 PlayCinematicRequest와 PlayAnchoredCinematic을 추가해 스킬 쪽에서 구조체 요청 또는 앵커 기반 편의 호출을 사용할 수 있게 했습니다.
- ACinematicTriggerActor에도 동적 Transform 옵션을 노출해 일반 컷신을 월드 고정, 트리거 기준, 플레이어 기준으로 선택할 수 있게 했습니다.
- Build.cs에 MovieSceneTracks 의존성을 추가해 Transform Origin 관련 엔진 헤더 경로를 안정화했습니다.

3. 요청 데이터 구조

필드	역할	대표 사용
Sequence	재생할 ULevelSequence 에셋입니다. 비어 있으면 요청은 실패합니다.	스킬 컷인, 궁극기, 레벨 컷신
InstigatorActor	시네마틱을 발동한 액터입니다.	스킬 사용자, 트리거에 들어온 플레이어
SubjectActor	연출의 중심 대상입니다.	플레이어, 타겟, 보스, NPC
AdditionalParticipants	명시적으로 멈추거나 반응시킬 추가 액터 목록입니다.	보스, 소환수, 주변 NPC
bAffectAllParticipants	true면 월드의 모든 CinematicParticipant를 수집합니다. false면 명시된 대상만 반응합니다.	궁극기 전체 정지, 특정 NPC만 정지
AnchorMode	시퀀스의 월드 원점을 어디서 계산할지 결정합니다.	플레이어 기준, 타겟 기준, 명시 트랜스폼
RotationSource	최종 시퀀스 회전을 어디서 가져올지 결정합니다.	플레이어 전방, 카메라 방향, 명시 회전
RelativeTransform	계산된 앵커 기준 로컬 오프셋입니다.	플레이어 앞 150cm에서 시작
bUseYawOnly	Pitch/Roll을 제거하고 Yaw만 사용합니다.	지상 액션 스킬 대부분

4. AnchorMode 해석

AnchorMode	계산 방식	권장 상황
AuthoredWorld	Level Sequence 에셋에 저장된 위치와 회전을 그대로 사용합니다. 동적 Transform Origin을 적용하지 않습니다.	정해진 맵 위치에서만 나오는 일반 컷신
InstigatorActor	InstigatorActor 또는 AnchorActor의 월드 트랜스폼을 원점으로 사용합니다.	플레이어 위치 기준 스킬 컷신
SubjectActor	SubjectActor 또는 AnchorActor의 월드 트랜스폼을 원점으로 사용합니다.	보스/타겟 중심 연출
InstigatorToSubject	Instigator 위치를 원점으로 잡고 Subject 방향을 바라보는 회전을 계산합니다.	플레이어가 타겟을 향해 쓰는 처형기/연계기
ExplicitTransform	요청자가 넘긴 ExplicitWorldTransform을 원점으로 사용합니다.	SkillComponent가 별도 계산한 위치/회전을 그대로 적용

5. RotationSource 해석

RotationSource	계산 방식	권장 상황
AnchorTransform	AnchorMode가 계산한 회전을 그대로 사용합니다.	일반적인 액터 기준 컷신
InstigatorActor	InstigatorActor의 월드 회전을 사용합니다.	캐릭터 정면 기준 공격
SubjectActor	SubjectActor의 월드 회전을 사용합니다.	타겟 전방/후방 연출
PlayerControlRotation	PlayerController의 ControlRotation을 사용합니다.	조준/입력 방향 기반 스킬
PlayerCameraRotation	PlayerCameraManager의 현재 카메라 회전을 사용합니다.	카메라가 보는 방향으로 나가는 스킬
ExplicitRotation	요청자가 넘긴 ExplicitRotation을 사용합니다.	SkillComponent가 카메라 yaw를 직접 계산해 의존성을 줄이는 방식

6. 동적 시퀀스 실행 흐름

- 1 요청자가 FCinematicPlaybackRequest를 생성합니다.
- 2 Director가 ULevelSequencePlayer와 런타임 ALevelSequenceActor를 생성합니다.
- 3 AnchorMode가 AuthoredWorld가 아니면 Director가 앵커 액터/소켓/명시 트랜스폼을 해석합니다.
- 4 InstigatorToSubject 모드라면 Instigator 위치에서 Subject 위치를 바라보는 회전을 계산합니다.
- 5 RotationSource가 지정되어 있으면 앵커 회전을 플레이어 회전, 카메라 회전, 명시 회전 등으로 교체합니다.
- 6 bUseYawOnly가 true면 Pitch/Roll을 제거해 지상 액션 게임에 맞는 Yaw 회전만 남깁니다.
- 7 RelativeTransform을 앵커에 곱해 최종 AnchorWorldTransform을 만듭니다.
- 8 LevelSequenceActor의 ActorTransform과 DefaultInstanceData.TransformOrigin에 최종 Transform을 적용합니다.
- 9 참가자들에게 OnCinematicStarted를 알리고 시퀀스를 재생합니다.
- 10 재생 종료 시 참가자 복구, ViewTarget 복구, 런타임 SequenceActor 정리를 수행합니다.

```
FinalRotation = ResolveRotation(Request, AnchorTransform)
AnchorTransform.SetRotation(FinalRotation)
FinalTransform = Request.RelativeTransform * AnchorTransform

LevelSequenceActor.SetActorTransform(FinalTransform)
InstanceData.TransformOrigin = FinalTransform
LevelSequenceActor.bOverrideInstanceData = true
```

왜 TransformOrigin이 필요한가

LevelSequenceActor의 액터 위치만 바꾸면 일부 카메라/스폰 액터에는 충분할 수 있지만, 시퀀서 안의 절대 Transform Section을 재사용 가능한 상대 연출로 만들려면 엔진이 평가 시 참조하는 Transform Origin을 넣어야 합니다. 이번 구현은 UDefaultLevelSequenceInstanceData를 통해 그 경로를 사용합니다.

7. 스킬 시퀀스 사용 패턴

목표	추천 설정	설명
플레이어 정면 기준 궁극기	AnchorMode=InstigatorActor, RotationSource=InstigatorActor, bUseYawOnly=true	캐릭터가 바라보는 방향으로 컷신 원점과 VFX가 정렬됩니다.
카메라 방향 기준 스킬	AnchorMode=InstigatorActor, RotationSource=PlayerCameraRotation, bUseYawOnly=true	플레이어 위치에서 시작하지만 방향은 현재 카메라 yaw를 따릅니다.
타겟을 향한 처형기	AnchorMode=InstigatorToSubject, RotationSource=AnchorTransform, SubjectActor=Target	플레이어 위치에서 타겟을 바라보는 방향으로 시퀀스가 놓입니다.
스킬 컴포넌트가 직접 계산	AnchorMode=ExplicitTransform 또는 RotationSource=ExplicitRotation	Cinematic 모듈이 플레이어 컨트롤러나 카메라 구현을 몰라도 됩니다.
플레이어 앞 오프셋	RelativeTransform.Location=(150, 0, 0)	계산된 앵커의 로컬 X축 앞쪽으로 150cm 이동한 위치에서 재생됩니다.
<pre> FCinematicPlaybackRequest Request; Request.Sequence = UltimateSequence; Request.InstigatorActor = Player; Request.SubjectActor = Target; Request.AnchorMode = ECinematicAnchorMode::InstigatorToSubject; Request.RotationSource = ECinematicRotationSource::AnchorTransform; Request.RelativeTransform = FTransform(FRotator::ZeroRotator, FVector(120.0f, 0.0f, 0.0f)); Request.bUseYawOnly = true; DirectorSubsystem->PlayCinematic(Request); </pre>		

8. 일반 컷신/트리거 사용 패턴

- 맵에 고정된 컷신은 AnchorMode를 AuthoredWorld로 둡니다. 시퀀서 에셋 안의 월드 위치가 그대로 사용됩니다.
- 트리거 액터 위치를 기준으로 재생하려면 ACinematicTriggerActor에서 bUseTriggerActorAsAnchor를 켜고 AnchorMode를 InstigatorActor 또는 SubjectActor가 아닌 앵커 기반 모드로 설정합니다. AnchorActor override가 우선 적용됩니다.
- 트리거에 들어온 플레이어 위치 기준으로 재생하려면 bUseTriggerActorAsAnchor를 끄고 AnchorMode=InstigatorActor를 사용합니다.
- 특정 NPC나 보스 기준 컷신은 ActivateCinematic(TargetActor)에 넘기는 대상과 SubjectActor/AnchorActor 설정을 분리해서 확장할 수 있습니다.

9. 의존성 분리 관점

모듈	알아야 하는 것	모르면 되는 것
SkillComponent	스킬 조건, 쿨타임, 게이지, 타겟, 원하는 앵커/회전 값	참가자 입력 잠금 방식, AI 정지 방식, 시퀀스 플레이어 내부
CinematicDirectorSubsystem	요청 데이터, LevelSequencePlayer, Transform Origin, 참가자 알림	스킬 쿨타임, 데미지 판정, 개별 캐릭터 복구 정책
CinematicParticipantComponent	시작/종료 컨텍스트와 자신의 옵션	요청자가 스킬인지 트리거인지, 시퀀서 카메라 컷 구성
CinematicTriggerActor	트리거 조건과 재생할 Sequence	플레이어 클래스의 내부 전투 컴포넌트 구조

카메라 회전 의존성을 더 줄이는 방법

RotationSource=PlayerCameraRotation을 쓰면 Director가 PlayerCameraManager를 읽습니다. 더 낮은 의존성을 원하면 SkillComponent가 Project_MJS의 카메라 규칙에 맞춰 yaw를 계산한 뒤 RotationSource=ExplicitRotation으로 넘기면 됩니다. 그러면 Cinematic 모듈은 카메라 구현을 거의 몰라도 됩니다.

10. 이번에 반영한 개선

개선 항목	반영 내용	에디터/코드 사용 지점
Binding Override	FCinematicBindingOverride를 추가해 Sequencer의 Binding Tag를 Player, Target, Weapon 같은 런타임 액터로 교체할 수 있게 했습니다. 트리거는 오버랩 액터를 Player 태그에 자동 바인딩할 수도 있습니다.	Request.BindingOverrides 또는 ACinematicTriggerActor의 BindingOverrides / bBindTriggeringActor
Input Lock Token	UCinematicInputLockSubsystem을 추가해 입력 잠금을 핸들 기반 Acquire/Release로 관리합니다. ParticipantComponent는 자신이 얻은 핸들만 해제합니다.	UCinematicParticipantComponent의 입력 잠금 처리
AI/Brain 정지 정책	bPauseAILogic 옵션을 추가해 AIController의 BrainComponent를 PauseLogic/ResumeLogic으로 멈추고 복구할 수 있게 했습니다.	Enemy/NPC에 붙은 UCinematicParticipantComponent
Network 정책	ECinematicNetworkPolicy를 추가해 LocalOnly, AuthorityOnly, AnyNetMode 중 어느 환경에서 재생할지 Request 단위로 결정합니다.	Request.NetworkPolicy 또는 트리거 액터의 NetworkPolicy
디버그 시각화	bDrawDebugAnchor, DebugDrawDuration, DebugDrawScale을 추가해 계산된 AnchorWorldTransform을 월드에 좌표축과 구체로 확인할 수 있게 했습니다.	Request.Debug 또는 트리거 액터의 CinematicDebug

11. 관련 코드 위치

항목	설명
Source/Project_MJS/Public/Cinematic/CinematicTypes.h	요청/컨텍스트 구조체, AnchorMode, RotationSource, BindingOverride, NetworkPolicy, Debug 옵션
Source/Project_MJS/Public/Cinematic/CinematicDirectorSubsystem.h	Director 인터페이스와 동적 Transform, BindingOverride, NetworkPolicy, Debug 헬퍼 선언
Source/Project_MJS/Private/Cinematic/CinematicDirectorSubsystem.cpp	LevelSequencePlayer 생성, Transform Origin 적용, BindingOverride 적용, 참가자 수집/알림/복구
Source/Project_MJS/Public/Cinematic/CinematicInputLockSubsystem.h	시네마틱 입력 잠금 핸들 서브시스템 선언
Source/Project_MJS/Private/Cinematic/CinematicInputLockSubsystem.cpp	AcquireInputLock/ReleaseInputLock 구현
Source/Project_MJS/Public/Cinematic/CinematicActionComponent.h	간단 호출, Request 직접 호출, 앵커 기반 편의 호출
Source/Project_MJS/Private/Cinematic/CinematicActionComponent.cpp	요청 기본값 보정, PlayerController 자동 해석
Source/Project_MJS/Public/Cinematic/CinematicParticipantComponent.h	입력/이동/Tick/AI/애니메이션 정지 옵션
Source/Project_MJS/Private/Cinematic/CinematicParticipantComponent.cpp	입력 잠금 핸들, AI Brain Pause/Resume, 기존 복구 흐름 구현
Source/Project_MJS/Public/Cinematic/CinematicTriggerActor.h	트리거 액터의 동적 Transform, BindingOverride, Network, Debug 설정 노출
Source/Project_MJS/Private/Cinematic/CinematicTriggerActor.cpp	트리거 요청 생성과 신규 옵션 전달
Source/Project_MJS/Project_MJS.Build.cs	LevelSequence, MovieScene, MovieSceneTracks, AIModule 등 시네마틱 모듈 의존성
운영 기준 기본 공격은 계속 AnimMontage와 Notify가 책임지고, 스킬/궁극기/컷신 연출만 이 Cinematic 구조를 사용합니다. 시퀀서는 전투 판정의 주인이 아니라 카메라, FOV, 포스트프로세스, 연출용 VFX를 제어하는 재생 레이어로 두는 것이 현재 Project_MJS 구조에 가장 안정적입니다.	